Computer Organization And Assembly Language

Lab Manual 04

Conditional Execution and Logical Operations in Assembly

Objectives:

- Understanding different types of jump instructions in MASM.
- Learning how to implement conditional execution using jumps.
- Exploring AND, OR, and XOR logical operations.
- Implement if-else equivalent structures in assembly.

Lab Instructor:

Mr. Tariq Mehmood Butt tariq.butt@pucit.edu.pk

Teaching Assistant:

Hafiz Muhammad Ahmad bcsf21m502@pucit.edu.pk

Lab Instructions:

1. Use MASM assembler for compiling and running programs.

2. Debug programs using DOS Debugger.
3. Document register changes at key points in the execution.
4. Verify results after execution.

Description of Various Jump Instructions

Jump instructions in assembly control the flow of execution based on conditions. These jumps rely on the status of specific flags set by previous instructions, such as CMP (compare), TEST or arithmetic operations.

Unconditional Jumps

· JMP label → Directly jumps to the specified label without any condition.

Conditional Jumps

Conditional jumps depend on the values of status flags in the FLAGS register:

Jump Instruction	Condition Checked	Flag(s) Involved
JE (Jump if Equal)	Jumps if operands are equal	ZF = 1
JNE (Jump if Not Equal)	Jumps if operands are not equal	ZF = 0
JG (Jump if Greater)	Jumps if first operand > second (signed)	ZF = 0, SF = OF
JL (Jump if Less)	Jumps if first operand < second (signed)	SF ≠ OF
JGE (Jump if Greater or Equal)	Jumps if first operand >= second (signed)	SF = OF
JLE (Jump if Less or Equal)	Jumps if first operand <= second (signed)	ZF = 1 or SF ≠ OF
JA (Jump if Above)	Jumps if first operand > second (unsigned)	CF = 0, ZF = 0
JB (Jump if Below)	Jumps if first operand < second (unsigned)	CF = 1
JAE (Jump if Above or Equal)	Jumps if first operand >= second (unsigned)	CF = 0
JBE (Jump if Below or Equal)	Jumps if first operand <= second (unsigned)	CF = 1 or ZF = 1
JC (Jump if Carry)	Jumps if carry flag is set	CF = 1
JNC (Jump if No Carry)	Jumps if carry flag is not set	CF = 0
JO (Jump if Overflow)	Jumps if overflow flag is set	OF = 1
JNO (Jump if No Overflow)	Jumps if overflow flag is not set	OF = 0
JS (Jump if Sign)	Jumps if sign flag is set (negative result)	SF = 1
JNS (Jump if No Sign)	Jumps if sign flag is not set (positive result)	SF = 0

Lab Tasks:

Task 1: Using Jumps for Comparison

1. Take two characters input.
2. Compare them using CMP.
3. Use JG, JL, or JE to store display whether the first number is greater, smaller, or equal to the second.

Task 2: Using Logical Instructions (AND, OR, XOR)

1. Initialize two 8-bit numbers in the .DATA segment.
2. Perform AND, OR, and XOR operations on them.
3. Store the results in registers and verify using the debugger.
4. Write input and output in the word file.

Task 3: If-Else Equivalent Implementation

1. Initialize an integer variable in the .DATA segment.
2. Check if the number is even or odd using bitwise operations.
3. Display “Even” or “Odd”.

Task 4: Manipulating case of letter.

1. Take a character input.
2. Display the same character in capital, small and opposite case.

Note: Manipulate the charater by using bitwise instructions.

Task 5: Bitwise Operations in Authentication System

1. Initialize a key variable in .DATA of length 5.
2. Initialize a dummy password variable of length 5 to take input.
3. Prompt user to enter password.
4. Perform a bitwise XOR operation between input and the predefined key.
5. Store the encrypted password in variable instead of original input.
6. Output key, encrypted password and original password by its decryption using XOR again.

Hint: XORing a number by the same key twice results the original number.

Task 6: Calculating the grade

1. Initialize a score value (0-100) in the .DATA segment.
2. Calculate grade according to FCIT's grading scheme.

3. Use CMP, JGE, and JL for if-else branching.

4. Print the grade.

Submission Guidelines:

· Submit your well-commented assembly source code.

Document the following for each task in a word file:

· Explanation of logic used.

· Encountered errors and how they were resolved.

Evaluation Criteria:

· Correctness and accuracy of the implemented assembly code.

· Proper use of registers and memory operations.

· Readability and commenting in the code.

· Completion and correctness of home tasks.

· Debugging approach and problem-solving skills applied.